

Exam-1 Program Verification 2021/2022

Megaron, 4th Oct 2021, 11:30 - 14:30

Lecturers: Wishnu Prasetya, Anna-Lena Lamprecht

1 Predicate Transformer [3pt]

1. [1 pt]

- (a) Give the weakest pre-condition of the statement below, with respect to the given post-condition. **Show** your wlp calculation; but **do not** simplify the resulting formula so that I can check your calculation.

```
{* ? *}
assume  $k \geq 0 \wedge y * x^k = \alpha^{1001}$ 
assume  $k > 0$ 
if  $k \bmod 2 = 0$  then { $x := x * x$  ;  $k := k/2$ }
                     else { $y := x * y$  ;  $k := k-1$ }
{*  $k \geq 0 \wedge y * x^k = \alpha^{1001} *$ } // post-cond
```

- (b) Consider the statement below. Give the weakest pre-conditions at position Q_2 and Q_1 , with respect to the indicated post-condition. The array **a** can be assumed to be of size $N > 0$. Show the wlp calculation. Do reduce/remove all **repy** sub-expressions from your results.

Be careful with your handling of $\forall$.

```
{ $Q_1?$ } assume  $(\forall j : 1 \leq j \leq N : a[j-1] \geq a[0])$ 
{ $Q_2?$ } assume  $1 \leq k \leq N$  ;
       $a[k] := a[k] - 1$  ;
       $k := k-1$ 
{*  $(\forall j : 1 \leq j \leq N : a[j-1] \geq a[0]) *$ } // post-cond
```

Just to remind you, the notation $(\forall j : P \ j : Q \ j)$ means the same as $(\forall j :: P \ j \Rightarrow Q \ j)$.

2. [0.8 pt] Consider a simple language of statements as in the language GCL from your project. Let us extend this language with a **switch**-statement with the following syntax:

```
switch  $e$  {
   $c_1 \rightarrow stmt_1$ 
  ...
   $c_K \rightarrow stmt_K$ 
}
```

where e is an integer-typed expression, each c_i is a *distinct* integer constant/literal, and $stmt_i$ is a statement. The **switch**-statement first evaluates e . If the result is equal to c_i , the whole sequence:

$stmt_i ; stmt_{i+1} ; \dots ; stmt_K$

will be executed. If e does not evaluate to any of the c_i , the whole statement simply does a **skip**.

An example is shown below:

```

switch x {
  4 → x := x - 2
  2 → x := x - 2
  0 → y := x
}

```

For example, if the switch expression x evaluates to 2, then $x := x - 2$; $y := x$ will be executed (so, both x, y end up with the value 0).

Your task: give a formula for constructing the **wlp** of a **switch**-statement.

3. [0.6 pt] A tool developer wants to build a program verification tool based on a post-condition transformer named **cp**. So, given a statement S and a post-condition Q ,

cp S Q

will construct a pre-condition. When given a specification:

$\{P\} S \{Q\}$

the tool first invokes **cp** S Q . Suppose this results in a predicate P' . The tool then checks the validity of $P \Rightarrow P'$. If the implication is valid, the specification $\{P\} S \{Q\}$ is accepted, and otherwise it is rejected.

The tool developer considers two variants of **cp**:

- Variant-1: a **cp** which is sound but *not* complete.
- Variant-2: a **cp** which is complete but *not* sound.

Questions:

- (a) Consider this result of **cp**:

cp $\underbrace{(\text{if } g \text{ then } x := x/2 \text{ else } x := x - 1)}_{\text{statement}} \underbrace{(x \geq 0)}_{\text{post-condition}}$

gives:

$$x/2 \geq 0 \vee x - 1 \geq 0$$

Question: which variant of **cp** was used? Explain your answer.

- (b) Name one main advantage and one main disadvantage (what could go wrong?) of each variant above.

4. [0.6 pt] Consider the following program, with the given post-condition. The array **a** is of size **N**. The programmer wants to verify that the program will not crash because it tries to access **a** at an invalid index, so he/she adds the **assert** statement in the loop.

```

{ * ? * }

while k < N do {
    assert 0 ≤ k < N ;
    sum := sum + a[k] ;
    k := k + 1 }

{ * 0 ≤ k * }

```

Show how you can calculate the **wlp** of this loop using fix point (FP) iterations, with respect to the given post-condition. Do the iterations terminate?

Hint: this property is obvious, but perhaps it is helpful to mention anyway: $(p \wedge g) \vee (p \wedge \neg g)$ is equivalent to just p .

2 Dealing with Loop [2 pt]

5. [1.5pt] Below are several small programs. They are valid with respect to the given pre- and post-conditions. For each program, give a **loop invariant** that would be good enough to prove the program's specification. You do not have to deliver a proof of this, but do consider your candidates carefully (e.g. simulate few runs to check them).

- (a) The following program calculates 2^{10} :

```

{ * k=0 ∧ x=1 * }

while k < 10 do { k := k+1 ; x := 2 * x }

{ * x = 210 * }

```

- (b) The following program calculates the minimum element of an array. The notation $\#a$ denotes the length of the array a .

```

{ * #a > 0 * }

m := a[0] ;
k := 1 ;
while k < #a do {
    if a[k] < m then m := a[k] else skip ;
    k := k+1 }

{ * (∀ i : 0 ≤ i < #a : m ≤ a[i]) * }

```

- (c) Below is a fragment of the Quicksort algorithm. The program takes the last element of the array a as a so-called *pivot* value. It then re-arranges the array a so that all elements $\leq pivot$ will be placed in the lower half of the array (and all elements $> pivot$ are thus placed in the array's upper half). The program **swap**(a, i, k) swaps the i -th

and k -th element of the array a .

$\{ * N = \#a \wedge N > 0 * \}$

```

pivot := a[N - 1] ;
i := 0; k := 0;
while k < N do {
  if a[k] ≤ pivot
    then swap(a, i, k) ; i := i + 1
    else skip ;
  k := k + 1}

```

$\{ * (\forall v : 0 \leq v < i : a[v] \leq \text{pivot}) \wedge (\forall v : i \leq v < N : \text{pivot} < a[v]) * \}$

6. [0.5pt] Consider the GCL language as in your project. Suppose we extend this language with a repeat-loop:

inv I **repeat** S **until** g

This statement will repeatedly do S until the condition g becomes true. Notice that S will thus be executed at least once. I is a loop-invariant that the programmer can annotate.

Suppose the programmer does provide an invariant, give a rule to calculate the **wlp** of the above **repeat**-construct. Do mention under which conditions your proposed wlp is sound.

3 Verification of Temporal Properties [5 pt]

7. [1pt] Consider a concurrent system of N processes $P_1 \dots P_N$, with $N \geq 3$. They work together using three resources r , s , and t . Each resource has the corresponding lock, e.g. $r.lock$ is the lock of r .

When a process P_i wants to use r , it first indicates this by setting a flag $req(P_i, r)$ to true. The process can use r once it has r 's lock. Let's also introduce the state predicate $has(P_i, r.lock)$ that is true if and only if P has r 's lock.

Formalize the requirements below in LTL or CTL.

- (a) It should **not** be possible for a process to lock all three resources.
 - (b) Whenever r is locked, eventually it will be free again.
 - (c) The system should be weakly fair with respect to the resource r . That is, whenever a process P_i requests r 's lock, and persistently maintains this request, it should eventually get the lock.
 - (d) When P_i is requesting the resource t , so setting the flag $req(P_i, t.lock)$ to true, it should be possible for P_i to backoff from this request while not getting the resource yet.
8. [1pt] Give for each LTL formula below, a Buchi automaton that equivalently describes the formula. Do not forget to specify what are the initial and accepting states are. Also, indicate whether you use a standard or generalized Buchi.

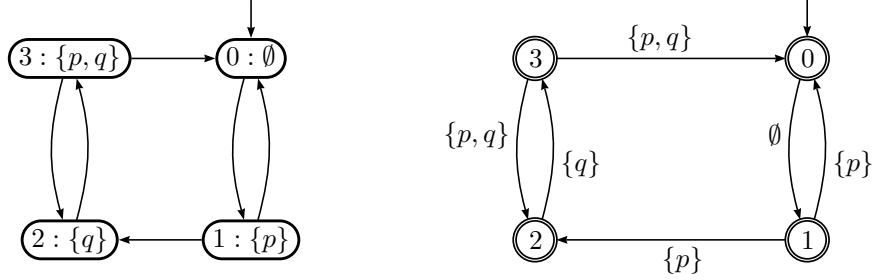
Assume $Prop = \{p, q\}$.

- (a) $p \mathbf{U} (q \mathbf{U} \Box \neg q)$

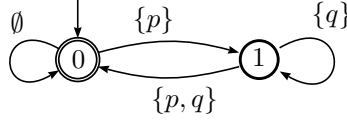
(b) $(\Box \Diamond p) \wedge (\Box \Diamond q)$

9. [2pt] Consider the following Kripke structure K (left picture) that you can think as modelling some program. $Prop = \{p, q\}$. The initial state is 0.

For your convenience on the right is a Buchi automaton that equivalently describes K (it accepts the same set of the sentences). All its states are marked as accepting.



- (a) Consider a second Buchi automaton B shown below. The state 0 is the initial state as well as its accepting state.



Construct the intersection automaton $K \cap B$.

- (b) **Describe the steps** of LTL model checking to verify whether the following LTL property ϕ holds on K :

$$\Box \Diamond \neg q$$

Do **provide your resulting intersection automaton**. You only need to construct it as far as the part that contains a counter example, if there is one. If there is no counter example, you will have to construct the full intersection automaton so that I can check that there is indeed no counter example.

- (c) **Describe the steps** of CTL model checking to verify whether the CTL property $\mathbf{AG}(\neg p \vee \neg q)$ holds on K . We can first observe that this property can also be expressed as this, in terms of CTL basic operators:

$$\psi = \neg \mathbf{E}(\mathbf{true} \mathbf{U} (p \wedge q))$$

10. [1pt] Let us consider the use of LTL for testing. Consider a non-deterministic program M . Let S be the set of all its possible states. This set can be infinite. The set of all possible executions of M may also be infinite. M has a single initial state, $s_{\text{start}} \in S$, and a single terminal state $s_{\text{end}} \in S$. When M enters s_{end} , it terminates.

A test suite V is a set of tests. Each test produces an execution σ of **finite** length. It may or may not end in s_{end} . Given an LTL formula ϕ , we want to be able to check it using V . We will do it as follows:

- (a) For each test $\sigma \in V$, we evaluate $\sigma \models \phi$. However, since σ is finite, we will change the standard definition of $\models$ so that the result is one of these three values:

- i. $\sigma \models \phi$ results $\top$ (true) if ϕ holds on all $\sigma ++ \tau$, where τ is any continuation of σ that M can produce.

Here, $++$ is a sequence concatenation operator.

- ii. $\sigma \models \phi$ results $\perp$ (false) if ϕ cannot possibly hold on all $\sigma ++ \tau$, where, as before, τ is any continuation of σ that M can produce.
- iii. $\sigma \models \phi$ results *undecided* if you cannot decide if ϕ would or would not hold on all $\sigma ++ \tau$ (e.g. because ϕ is a future formula and σ is not long enough to decide if ϕ would hold or otherwise).

- (b) ϕ is rejected if there is one $\sigma \in V$ where $\sigma \models \phi$ results in $\perp$.

ϕ is accepted if it is not rejected, and there is at least one $\sigma \in V$ where $\sigma \models \phi$ results in $\top$.

Else the result of the verification is undecided.

Your task: give a formal definition of this variation of the evaluation function $\sigma \models \phi$ for LTL formulas. The syntax of LTL is given below. Your definition of $\sigma \models \phi$ should be decidable in finite number of steps.

Hint: define it through $\sigma, i \models \phi$, and recursively over the structure of ϕ .

$$\begin{array}{lcl} \phi & ::= & p \text{ , where } p \text{ is a state predicate} \\ & | & \neg \phi \\ & | & \phi \wedge \psi \\ & | & \mathbf{X} \phi \\ & | & \phi \mathbf{U} \psi \end{array}$$

A 'state predicate' is a predicate that can be evaluated directly on any $s \in S$ to know if it holds there or not.